# 论高可用架构的设计与实现

> ⚠ **关于本文中的数字** 本文的量化数据（可用率、响应时间、故障恢复时长等）均为**示例值**，用于展示论文应有的完整度与写法。 **架构类数字（表数量、接口数、配置值）已从项目源码核实，可直接使用**； **业务规模类数字（门店数、用户数、业务量、改善幅度）在项目资料中尚未确定，属示例值，背诵前请替换为你的真实数据**。 文末附有「待替换数字清单」，便于逐项核对。

## 试题

**论高可用架构的设计与实现**

高可用架构是指通过冗余设计、故障转移、容错处理、资源隔离等技术手段，使系统在部分组件发生故障时仍能持续对外提供服务，并将故障对业务的影响控制在可接受范围内。高可用架构设计需要明确系统的可用性目标，识别影响可用性的关键因素，并在成本与可用性之间做出合理权衡。

请围绕"高可用架构的设计与实现"论题，依次从以下三个方面进行论述：

1. 简要叙述你参与设计和开发的高可用系统，以及你所担任的工作；
2. 详细论述你在该系统中采取了哪些措施来保障系统的高可用性；
3. 分析你在可用性保障过程中遇到的问题及解决措施，并说明如何验证系统的可用性。

## 摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台为例，论述了高可用架构的设计与实现。该平台服务集团营业本部与 300 余家门店的约 5000 名业务人员，业务具有强季节性——长假前预约量激增，平峰期迅速回落，对可用性提出刚性要求。我担任系统架构师，负责可用性目标制定、冗余与故障转移方案设计、资源隔离与可靠性机制建设。针对基础设施单点、资源争抢、消息重复推送、会话不一致等风险，我按分层设防思路设计：基础设施层以多可用区、健康检查与数据库主备实现冗余与自动转移；应用层以独立读写数据源和部署单元隔离故障域；以集中式会话存储与弹性伸缩支撑无状态扩容；数据与业务层以只读副本、消息幂等键、专用重试与死信队列保障数据可靠、控制重复。上线 8 个月累计可用率达到 99.95%，批处理在 4 小时 47 分内完成，重复发送率降至 0.01% 以下，并通过压测、故障注入与监控验证了方案。

## 正文

### 一、项目背景与可用性目标

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务 H.I.S. 集团 300 余家营业门店与约 5000 名业务人员，日均处理预约约 3 万笔、消息与通知约 12 万条。我在项目中担任系统架构师，负责可用性目标制定、冗余与故障转移方案设计以及可靠性保障机制的建设。

旅游预约一旦不可用，客户会转向其他渠道，流失订单通常无法挽回；消息漏发或重复则直接影响客户体验与品牌信誉。因此，我与业务方、运维方共同定义目标：年可用率不低于 99.9%；全部事务 90 分位响应时间不超过 5

秒，预约查询等核心场景收紧至 2 秒以内；夜间批量作业须在次日营业前完成；任何单一组件故障不得导致全站不可用；核心数据 RTO 不超过 30 分钟、RPO 不超过 5 分钟。

我们没有追求 99.99%：跨区域多活成本约为 99.9% 的数倍，而旅游预约在 99.9% 下的损失已可接受。因此把资源优先投入隔离、转移与可观测性，而非单纯堆高冗余等级。

在基础设施由云平台多可用区托管、硬件故障已被平台兜底的前提下，我首先确认了基础设施层的冗余与切换路径；在此基础上，我认为应用层最大的威胁来自内部三类问题：资源争抢、重复处理、状态耦合。下面按分层设防展开。

## 二、高可用架构的分层设计

我的设计按四层展开：基础设施层解决"单点"，应用层解决"故障扩散"，状态层解决"扩容与一致性"，数据与业务层解决"数据不丢、失败可恢复"。

**第一层：基础设施冗余与故障转移。** 平台部署于 AWS 东京区域。Web 与 Service 以多实例分布在至少两个可用区，负载均衡器通过健康检查自动摘除异常实例；关系型数据库启用 Multi-AZ 主备，主库故障时自动切换；会话存储采用 Redis Cluster（6 节点），分片与主从提供故障转移；S3 与 SQS 由云平台跨可用区持久化；数据库每日备份并开启时间点恢复，支撑 RTO≤30 分钟、RPO≤5 分钟。我按请求链路检查单点：入口、应用、数据库、会话、文件与队列均有替换路径；区域级故障以备份与跨区预案兜底。

**第二层：资源与故障域隔离。** 这是本项目投入最多的一层。成本文件生成、外部预约导入等批量作业需从生产库大量读取并回写，而在线预约接口同时访问同一数据库。若共享连接池，夜间批处理峰值会推高在线响应时间，甚至耗尽连接。为此，我为批处理配置独立读写数据源：读走只读副本，写走主库；两套连接池独立并设上限，Mapper 按 repository/reader 与 repository/writer 包路径隔离。上线前压测中，批量作业期间在线查询曾由 300 毫秒劣化至 2.5 秒；改造后稳定在 320 毫秒以内，批量作业耗时反而下降约 30%。同时把隔离级别设为读已提交，减少长事务持锁；批量游标限制在 200 条，防止结果集一次性载入导致内存溢出。

部署上，系统按 Web、Service、Batch、FaaS 四类单元划分。Batch 不启用 Web 容器，与在线业务无进程级耦合；14 个 Lambda 按业务域拆分，由 SQS、DynamoDB Stream 与 EventBridge 触发，单域异常不会波及他域。代价是运维单元增多，但把"一个模块崩溃导致全站不可用"降为"单个业务域受影响"。

**第三层：状态外置与弹性伸缩。** Web 层多实例后，若会话存在本地内存，负载均衡会把同一用户分发到不同实例，导致反复掉线。我引入 Spring Session Data Redis，把会话集中到 Redis Cluster，超时设为 28800 秒——8 小时恰好覆盖营业人员一个工作日，避免营业时段因会话过期中断。会话外置后 Web 实例完全无状态，可任意扩容与重启。

针对强季节性，Web 与 Service 按 CPU 与请求数自动伸缩；消息先入 SQS 再由 Lambda 消费，削峰填谷；预约查询走只读副本，避免峰值读流量压到主库。长假前做容量评估与扩容预演。

**第四层：数据层可靠性与业务容错。** 数据层对需要并发控制的实体采用乐观锁配合重试；把护照、文档等文件从数据库剥离到 S3，降低库体积与备份恢复时长；为集团主数据建立本地只读副本，使外部平台故障不影响业务查询。批处理的关键参数（执行基准日、取得范围、导入类型）显式传入，不依赖系统时钟，使补数与重跑可复现。

业务容错方面，外部调用均设连接与读取超时；重试由专用服务承担，最大 4 次、间隔 1 秒，超过阈值进入死信队列并告警；接入层设连接数与并发上限，Lambda 配置预留并发，防止单域耗尽资源；非核心链路采用"先落库、后补发"的降级策略。业务级熔断尚未引入，当前以云平台并发上限与队列缓冲兜底，后续补齐。

## 三、关键技术的实现

**幂等控制，把重复推送控制在可接受范围。** 消息与邮件推送是本平台最典型的可靠性敏感点，一旦重复发送就是体验事故。我设计了三层防护。第一层是作业级防重：作业管理表记录执行状态，同一作业处于"处理中"时重复触发会被拒绝；批处理触发另设 10 分钟幂等时间窗，按业务键去重，防止调度器重复拉起。第二层是消息级幂等：每条消息发送前生成全局唯一消息 ID，与业务键一起落库并加唯一约束；调用网关时携带该 ID 作为幂等键；投递状态细化为"待投递/投递中/已投递/未知/失败"，超时后先按消息 ID 对账，确认未投递才重试。第三层是专用重试服务：重试逻辑与业务解耦，最大 4 次、间隔 1 秒，超过阈值进入死信队列并告警，使重试策略可独立调整。不支持幂等键的网关，采用"至少一次投递 + 下游定时对账"兜底。

**可观测性建设。** 高可用的前提是故障能被及时发现。批处理是短生命周期进程，作业结束即退出，常规进程监控采集不到状态。我让作业主动把指标推送到 Prometheus PushGateway，并把关闭动作设为"进程退出前投递"，确保最后一批指标不丢失；函数侧用 AWS Lambda Powertools 输出结构化日志与链路追踪。在此基础上，把可用性目标拆成接口错误率、P90/P99 延迟、健康实例数、队列积压、批处理完成时刻与消息重复率等监控项，并设分级告警，使批量失败、依赖异常与实例摘除都能第一时间被发现。

**发布与回滚。** 变更过程同样影响可用性。我们采用滚动发布：先更新少量实例，观察错误率与延迟后再全量；数据库变更遵循"先兼容、后切换"，新增字段先允许为空，待新旧版本并存期结束后再收紧约束；配置与凭证通过 SSM Parameter Store 管理。每次发布保留上一版本镜像与回滚脚本，异常可在分钟级回退；发布本身也纳入变更评审与演练。

## 四、实施中的问题及解决措施

**问题一：消息重复推送。** 上线初期，客户反馈同一封通知邮件收到两次。排查发现，邮件网关超时后系统自动重试，而首次请求实际已成功。本质是分布式环境下的幂等性问题：作业级防重只能阻止重复触发，无法判断超时投递是否已成功。解决方案即第三节的三层防护，核心是给每条消息唯一 ID，超时后先对账再决定是否重试。改造后重复发送率由约 0.3% 降至 0.01% 以下。

**问题二：主库故障切换期间的写入失败。** 上线后第三个月，主库所在可用区网络抖动，RDS Multi-AZ 自动切换，切换期间部分写入失败，个别 Web 实例被健康检查摘除。多实例与状态外置使已登录用户未被强制下线，读请求由只读副本承接；告警在 3 分钟内触发，切换完成后恢复，本次业务恢复约 18 分钟。复盘后补充两项措施：统一数据库连接超时与重试配置；把主备切换纳入监控与季度故障演练。冗余只有经过演练才是真正可用的。

**问题三：多实例部署下的会话丢失。** 平台随业务量增长横向扩容 Web 实例后，用户频繁被强制重新登录。原因是会话存在单实例本地内存，请求被分发到其他实例时无法识别。引入 Spring Session Data Redis、把会话集中到 Redis Cluster 后解决；会话集中在服务端还带来安全收益，可强制失效任意会话，实现管理员踢出用户、改密后全端下线。

**关于可用性的验证方式。** 我们建立了覆盖开发、上线、运行三阶段的验证体系。第一，明确指标口径：可用率按"1 - 不可用时长/统计周期"计算，以分钟级健康检查与接口错误率为判据，同时跟踪 MTBF、MTTR、P90/P99、消息重复率与批处理完成时刻。第二，开发阶段用 Testcontainers 与 LocalStack 搭建与云端一致的依赖环境，对读写分离、幂等、会话外置编写集成测试。第三，上线前做压力测试与故障注入：压测覆盖长假峰值与批处理/在线并发；注入包括杀实例、断依赖、触发主备切换，验证摘除、切换与恢复。第四，运行期用 Prometheus PushGateway、Grafana 与 CloudWatch 持续观测，把目标拆成 SLO 告警，每季度演练并复盘。上线 8 个月累计可用率 99.95%，期间 5 次计划外故障累计不可用约 2.9 小时，平均恢复约 35 分钟；批处理均在 5 小时内完成（实测 4 小时 47 分），未发生因批量作业导致的在线降级。

### 五、总结与展望

平台上线至今约 8 个月，累计可用率 99.95%，超过 99.9% 的目标；批处理均在次日营业前完成，未发生因批量作业导致的在线降级；消息重复发送率控制在 0.01% 以下。

仍有三点不足：一是跨存储最终一致性链路缺少统一延迟监控，排查路径偏长；二是低频 Lambda 存在冷启动延迟，尚未配置预置并发；三是重试为固定间隔，依赖持续故障时可能放大压力，业务级熔断与精细化限流尚未引入，流量突增时主要依靠云平台并发上限与队列缓冲兜底。

后续计划引入一致性延迟监控、预置并发、指数退避与抖动、服务端熔断限流，并定期开展故障演练。

通过本项目，我认识到高可用不是靠某一项技术实现的，而是由一系列设计构成的体系：用多可用区与自动切换消除基础设施单点，用资源隔离消除内部争抢，用故障域隔离限制故障扩散，用状态外置支撑水平扩容，用幂等与重试应对不可避免的失败。更重要的是，高可用必须有明确的目标与验证手段——没有量化目标的"高可用"只是口号，无法验证的高可用则无法证明其存在。

---

## 写作提示（不计入论文字数，背诵时请忽略本节）

### 待替换数字清单

| 位置 | 示例值 | 请替换为 |
| --- | --- | --- |
| 门店数 / 业务人员数 | 300 余家 / 约 5000 名 | 真实数量 |
| 日均预约量 / 消息量 | 约 3 万笔 / 约 12 万条 | 真实数量 |
| 年可用率目标 | 99.9% | 真实指标 |
| 上线 8 个月累计可用率 | 99.95% | 真实指标（与其余 8 篇统一口径） |
| 计划外故障次数 / 累计不可用 / 平均恢复 | 5 次 / 约 2.9 小时 / 约 35 分钟 | 真实运维数据 |
| 本次主备切换恢复时间 | 约 18 分钟 | 真实数据 |
| 响应时间目标 | 90 分位 5 秒以内（核心场景 2 秒） | ✔ 已核实 |
| 压测劣化情况 | 300ms → 2.5s | 真实压测数据 |
| 改造后响应时间 | 320 毫秒以内 | 真实压测数据 |
| 批量作业耗时下降 | 约 30% | 真实数据 |
| 批处理完成时限 / 实测 | 5 小时以内 / 4 小时 47 分 | ✔ 已有测试报告 |
| 重复发送率 | 0.3% → 0.01% | 真实数据 |
| 会话超时 | 28800 秒（8 小时） | ✔ 已核实 |
| 重试策略 | 最大 4 次 / 间隔 1 秒 | ✔ 已核实 |
| 批量游标 | 200 条 | ✔ 已核实 |
| RTO / RPO | 30 分钟 / 5 分钟 | 真实指标 |

### 口径统一提醒（重要）

1. **可用率**：其余 8 篇写作「年可用率 99.95%」，但项目只运行约 8 个月。本篇已改为「上线 8 个月累计可用率」，建议全篇统一；若考卷要求写年度指标，可写「折合年化约 99.95%」。
2. **会话存储**：素材库与模板写「Spring Session Data Redis + Redis Cluster（6 节点）」、超时 3600 秒；本篇按索引中的核实值写 28800 秒。两处需统一，避免同一项目在不同论文中数字打架。
3. **只读副本**：本篇写「读数据源指向只读副本」。若你的实际项目只是同一数据库的只读账号，请改为「只读账号 + 独立连接池」，并删除「物理隔离」类表述，否则技术上不严谨。
4. **熔断限流**：本项目未见 Sentinel/Hystrix/Resilience4j 证据，本篇只写「云平台并发上限与队列缓冲兜底」，不要改写成「已使用熔断组件」。同理，若项目未实际配置预留并发、死信队列或自动伸缩，请改为「规划中」或删除，避免被追问。

### 本文字数

摘要约 315 字，正文约 3065 字，**合计约 3380 字。**若按「合计 2500～3000 字」的严格口径，可删去「发布与回滚」一节（约 150 字）与问题二的复盘细节（约 100 字），即可压到 3000 字以内。

### 可复用要点

- **开头必写「可用性目标」**——这是高可用类论文的标准起手式，没有目标的论文会被判定为空谈。
- **用「分层设防」做总纲**——基础设施冗余、资源与故障域隔离、状态外置与弹性、数据可靠与业务幂等四层，能覆盖题目定义里的冗余、故障转移、容错、资源隔离四项手段。
- **「在本项目中，基础设施单点由云平台兜底，应用层最大的威胁来自内部资源争抢」**——比直接说「最大威胁来自内部」更严谨，也更有洞察力。
- **幂等要区分「作业级防重」与「消息级幂等」**——只写作业锁会被追问「网关超时但实际成功怎么办」，补上消息唯一 ID 与对账才闭环。
- **验证要写四件事**——指标口径（可用率、MTBF/MTTR）、集成测试（Testcontainers/LocalStack）、故障注入（杀实例、断依赖、主备切换）、运行监控（Prometheus/Grafana/CloudWatch）。
- **结尾主动写不足**——固定间隔重试、冷启动、未引入熔断限流，既真实又体现架构判断力；但要用「云平台兜底 + 后续补齐」的方式写，不要只写「没有」。